HIREFLOW

Intelligent College Placement Management & Recruitment Platform

Complete End-to-End Development Blueprint

From requirements and architecture to database, APIs, three role-based workflows, AI, testing, Docker, deployment, Git recovery and a 30-day execution plan.

Stack: React + Vite • Node.js + Express • MongoDB • Python + FastAPI • JWT • Embeddings • Docker • Git/GitHub

Core users: Student • Placement Officer • Recruiter

Version 1.0 • September 2026

1. Project Vision and Problem

Problem: College placement operations are often spread across spreadsheets, messages, email, resumes and manual lists. Placement officers spend time checking eligibility and tracking applications; students struggle to see opportunities and status; recruiters receive large applicant pools without a unified workflow.

Solution: HireFlow centralizes the complete placement lifecycle. It combines deterministic placement rules with AI-assisted resume/job matching, while keeping Student, Placement Officer and Recruiter workflows separate.

Project positioning: This is primarily a full-stack business application with an AI service—not merely an AI chatbot.

What the system solves

- Centralized student, company and placement-drive data
- Automatic eligibility checking
- Application and interview tracking
- Recruiter candidate search and shortlisting
- Resume parsing and semantic job matching
- Student skill-gap identification
- Placement-cell analytics and reporting

End-to-end flow

Recruiter/Officer creates opportunity → Officer publishes drive → eligibility engine identifies students → students view/apply → officer monitors/approves when configured → recruiter reviews → AI assists matching → shortlist → assessment/interview → selection → officer closes drive → analytics.

2. Users and Responsibilities

Role	Main question	Core capabilities
Student	What can I apply for and how do I improve?	Profile, resume, eligible drives, applications, interviews, AI match, skill gap, preparation
Placement Officer	How do I coordinate the college process?	Students, companies, drives, eligibility, approvals, applications, schedules, analytics
Recruiter	Which candidates should I review and hire?	Jobs, applicants, filters, candidate details, shortlist, interviews, selection

Placement Officer is a first-class role

The officer is the bridge between students and companies. The platform should allow configurable application policy: **DIRECT** or **OFFICER_APPROVAL**. This makes the product realistic for different colleges.

3. Functional Requirements

ID	Requirement	Definition of done
FR-01	Authentication	Register/login/me, password hashing, JWT, protected routes
FR-02	RBAC	Student/Officer/Recruiter permissions enforced on server
FR-03	Student profile	Academic details, skills, projects, certifications, resume
FR-04	Company management	Create, verify, update and view company
FR-05	Placement drives	Create, publish, close, archive, configure rules
FR-06	Eligibility	CGPA, branch, year, backlog and configurable rules
FR-07	Applications	Apply, review, shortlist, reject, withdraw where allowed
FR-08	Interviews	Schedule rounds, time, mode, room/link and status
FR-09	Resume	Upload, extract text, parse structured data
FR-10	AI matching	Semantic similarity + explicit matched/missing skills
FR-11	Skill gap	Missing skills and target learning areas
FR-12	Preparation	Resume/job-grounded interview questions
FR-13	Notifications	Drive/status/interview notifications
FR-14	Analytics	Funnel, selections, company/drive statistics
FR-15	Audit	Important officer/recruiter actions recorded

4. Non-Functional Requirements

Area	Requirement
Security	Hashed passwords, JWT expiry, RBAC, validation, secure file handling, secrets outside Git
Performance	Pagination, indexes, efficient queries, controlled API calls
Reliability	Central errors, logs, health checks, graceful AI failure
Maintainability	Modular services/controllers/models, documented APIs
Scalability	Stateless API design; AI isolated as a separate service
UX	Responsive pages, validation, loading/error/empty states
Data quality	Duplicate prevention, valid status transitions, consistent academic data
Recovery	Git checkpoints plus AI_CONTEXT/PROGRESS documentation

5. Technology Stack

Layer	Technology	Use
Frontend	React + Vite	Role dashboards and workflows
UI	Tailwind CSS or chosen component system	Consistent responsive interface
Backend	Node.js + Express	REST API and business rules
Database	MongoDB	Placement entities and flexible profiles
Auth	JWT + bcrypt	Authentication and authorization
AI service	Python + FastAPI	NLP/ML isolation
NLP	Sentence Transformers / embeddings	Semantic resume-job similarity
Testing	Jest/Supertest + frontend tests	Automated verification
API docs	Swagger/OpenAPI	Interactive API contract
DevOps	Docker + Compose	Repeatable environments
Version control	Git/GitHub	History, checkpoints and recovery

Do not add technologies just to make the stack look bigger. A clean modular monolith plus a separate AI service is enough for this project.

6. System Architecture

Logical architecture

```
Browser / React UI
↓ REST/HTTPS
Node.js + Express API
■■■ Authentication + RBAC
■■■ Student / Officer / Recruiter modules
■■■ Drives + Eligibility
■■■ Applications + Interviews
■■■ Notifications + Analytics
■■■ AI orchestration
↓
MongoDB

Node API → Python FastAPI AI Service → Resume parsing / embeddings / matching / skill-gap / question generation
```

Keep core business rules in Node.js. Keep AI/ML operations in Python. Add /health endpoints to both services.

7. Repository Structure

```
HireFlow/
■■■ frontend/src/{components,pages,layouts,hooks,services,context,routes,utils}
■■■ backend/src/{config,controllers,middleware,models,routes,services,validators,utils}
■■■ ai-service/app/{routes,services,models,utils}
■■■ docs/{REQUIREMENTS,ARCHITECTURE,DATABASE,API,AI_CONTEXT,PROGRESS,DECISIONS}.md
■■■ docs/tasks/TASK-XXX.md
■■■ docker/
■■■ docker-compose.yml
■■■ README.md
■■■ .gitignore
```

The repository—not the Copilot conversation—must contain the project's memory.

8. Database Design

Collection	Key fields
users	name, email, passwordHash, role, status, createdAt
students	userId, collegeId, branch, cgpa, graduationYear, backlogs, skills, projects, certifications, resumeId
companies	name, description, website, industry, location, contacts, verificationStatus
jobs	companyId, title, description, skills, package, locations, status
placementDrives	jobId, createdBy, eligibilityRules, applicationMode, startDate, endDate, status
applications	driveId, studentId, companyId, status, appliedAt, matchInfo
resumes	studentId, fileRef, extractedText, parsedProfile, version
interviews	applicationId, round, scheduledAt, mode, locationOrLink, status
assessments	driveId, applicationId, type, score, status
notifications	userId, type, title, message, readAt
auditLogs	actorId, action, entityType, entityId, metadata, createdAt

Indexes

- users.email unique
- students.branch + graduationYear + cgpa
- jobs.status + companyId
- placementDrives.status + startDate
- applications.driveId + status
- applications.studentId + createdAt
- interviews.scheduledAt

9. Authentication and Authorization

1. Register → validate → hash password → create user.
2. Login → verify password → issue JWT.
3. Protected request → authenticate middleware → authorize role → controller/service.
4. Never trust a role supplied by the frontend. Enforce authorization on the server.

Permission matrix

Action	Student	Officer	Recruiter
Own profile	CRUD	View/verify	View applicant
Create drive	No	Yes	Yes/configurable
Publish drive	No	Yes	Configurable
Apply	Yes	Only if explicitly supported	No
Review applications	Own	Yes	Own company
Change hiring stage	No	Coordinate	Yes
College analytics	No	Yes	No

10. Placement Officer Module

The Placement Officer should have the most complete operational workflow.

- Dashboard: students, companies, active drives, applications, shortlisted, selected
- Student management: search, verify profile, view academic data
- Company management: register/verify company and contacts
- Drive management: create/edit/publish/close/archive
- Eligibility view: eligible/ineligible/applied/shortlisted counts
- Application monitoring and approval
- Interview calendar and coordination
- Final placement records and reports
- College-wide analytics and exports

Officer workflow

1. Create/verify company.
2. Create or select job.
3. Configure eligibility and application mode.
4. Publish drive.
5. Monitor eligible population and applications.
6. Approve/forward applications when required.
7. Coordinate interviews/assessments.
8. Record final outcomes.
9. Close drive and review analytics.

11. Student Module

- Dashboard
- Profile and profile completion
- Resume upload/versioning
- Placement drives
- Eligibility and job details
- Apply/withdraw where permitted
- Application timeline
- Interview schedule
- AI skill gap
- Interview preparation
- Notifications

A student should see **why** they are eligible/ineligible and, where available, what skills are missing.

12. Recruiter Module

- Recruiter dashboard
- Company profile
- Create/manage jobs
- Applicant search/filter
- Candidate detail
- Resume view
- AI match information
- Shortlist/reject
- Interview rounds
- Selection outcome

AI matching should be an assistive signal. Keep deterministic eligibility and human review separate from AI similarity.

13. Placement Drive and Eligibility Engine

```
Example rule object:
minCgpa: 7.0
allowedBranches: [CSE, ISE, AIML, CSE-DS]
maxBacklogs: 0
graduationYear: 2027
applicationMode: OFFICER_APPROVAL

Example result:
eligible: true
reasons: []
missingSkills: ["AWS"]
checkedRules: [...]
```

Important: eligibility is deterministic. AI matching is a separate layer. A student can be eligible but have a lower semantic match.

14. Applications and Interviews

```
Application lifecycle:
APPLIED → UNDER_REVIEW → SHORTLISTED → ASSESSMENT → INTERVIEW → SELECTED
Terminal alternative: REJECTED
Student-controlled where allowed: WITHDRAWN
```

Implement a backend transition map so clients cannot arbitrarily change statuses.

Interview fields

- applicationId
- round
- scheduledAt
- mode
- room/link
- interviewer/participants where permitted
- status
- feedback with access control

15. Resume Management

Pipeline: upload PDF → validate type/size → secure storage → extract text → normalize → parse fields → save structured profile → use for matching.

- Keep original file reference
- Store extracted text and parsed data separately
- Store parser version
- Support resume versions
- Show clear extraction failure/retry state

Do not make uploaded resumes public by default.

16. AI/ML Service

The Python service should expose focused endpoints:

Endpoint	Input	Output
POST /parse-resume	PDF/text	Structured profile
POST /match-resume	Resume + job	Similarity + matched/missing skills
POST /skill-gap	Skills + target job	Missing skills + targets
POST /generate-questions	Resume + job + difficulty	Grounded question set

AI architecture

Node backend authenticates the user and validates access → Node calls FastAPI → FastAPI runs parsing/embedding/model logic → result returned → Node applies business rules → frontend displays result.

17. Resume-Job Matching

1. Normalize resume and job text.
2. Extract explicit skills.
3. Create embeddings for relevant text.
4. Calculate semantic similarity.
5. Compare explicit skills.
6. Combine signals into an explainable result.
7. Return matched skills, missing skills and score.

Example:
Match: 78%
Matched: Java, SQL, React
Missing: AWS, Docker
Eligibility: Eligible

Create a small labeled evaluation set during development. Report actual evaluation results; never invent accuracy.

18. Skill Gap and Interview Preparation

Skill gap: compare current student skills with target role requirements and show missing skills. Interview preparation: generate technical, project, role-specific and behavioral questions from the resume/job context.

Add validation so the AI does not invent projects, certifications or experience that are not present in the input.

19. Notifications

Event	Recipient	Example
Drive published	Eligible students	New drive available
Application status changed	Student	You have been shortlisted
New application	Officer/Recruiter	New candidate application
Interview scheduled	Student	Interview scheduled
Interview changed	Participants	Time/room changed
Deadline approaching	Relevant students	Drive closes soon

Start with in-app notifications. Email can be added after the core system works.

20. Analytics

Placement Officer

- Registered/verified students
- Active companies
- Active drives
- Applications
- Shortlisted
- Selected
- Placement rate with a clearly defined denominator
- Company-wise selections
- Drive funnel
- Skill demand

Recruiter

- Open jobs
- Applications
- Shortlisted
- Interviews
- Selections

Student

- Profile completion
- Applications
- Shortlisted
- Upcoming interviews
- Skill gaps

21. REST API Plan

Area	Representative endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Students	GET/PUT /api/students/profile, GET /api/students
Companies	POST/GET/PUT/DELETE /api/companies
Jobs	POST/GET/PUT/DELETE /api/jobs
Drives	POST/GET/GET/:id/PUT/DELETE /api/drives
Eligibility	GET /api/drives/:id/eligibility
Applications	POST /api/applications, GET /api/applications/my, GET/PATCH /api/applications/:id
Interviews	POST/GET/PATCH /api/interviews
Notifications	GET /api/notifications, PATCH /api/notifications/:id/read
Analytics	GET /api/analytics/officer|recruiter|student
AI proxy	POST /api/ai/parse-resume, /match-resume, /skill-gap, /questions

Use consistent status codes, request validation, predictable response shapes, pagination and OpenAPI documentation.

22. Frontend Implementation

- Role-aware routing
- ProtectedRoute and RoleRoute
- Reusable form/input/table/modal components
- Axios service layer
- Loading/error/empty states
- Responsive dashboards
- Search/filter/pagination
- Application status timeline
- Charts for analytics

Student pages

Login → Register → Dashboard → Profile → Resume → Drives → Job Detail → Applications → Interviews → Skill Gap → Preparation → Notifications

Officer pages

Dashboard → Students → Companies → Drives → Drive Detail → Eligibility → Applications → Interviews → Reports

Recruiter pages

Dashboard → Company → Jobs → Applicants → Candidate Detail → Shortlist → Interviews → Selection

23. Backend Implementation

Recommended request flow: Route → validation → authentication → authorization → controller → service/business logic → model/database → response.

Keep controllers thin. Put eligibility, application transitions and AI orchestration in services. Use centralized error middleware.

Backend modules

- auth
- students
- companies
- jobs
- drives
- eligibility
- applications
- interviews
- notifications
- analytics
- AI integration
- audit

24. Testing Strategy

Test layer	Examples
Unit	Eligibility rules, status transitions, validators
API integration	Auth, role restrictions, jobs, applications
Frontend	Protected routes, critical forms, application flow
AI evaluation	Resume parsing and matching on fixed sample set
End-to-end	Student → drive → apply → recruiter → shortlist → interview → select

Critical negative tests

- Wrong password rejected
- Student cannot access officer API
- Duplicate application rejected
- Ineligible application handled according to drive policy
- Recruiter cannot modify another company's candidate
- Invalid status transition rejected
- Malformed resume gives clear error
- AI service outage produces graceful fallback

25. Security Checklist

- Hash passwords
- Short-lived/rotated JWT strategy as appropriate
- Server-side RBAC
- Validate and sanitize input
- Restrict upload type/size
- Secure resume storage
- HTTPS in production
- Secrets in environment/secret manager
- Careful CORS
- Rate-limit auth endpoints
- Prevent cross-user data access
- Audit administrative actions
- No stack traces/secrets in production responses

26. Docker and Deployment

```
Local Compose target:
frontend
backend
ai-service
mongodb (optional local database container)
```

```
Backend env:
PORT=
MONGODB_URI=
JWT_SECRET=
AI_SERVICE_URL=

Frontend:
VITE_API_URL=

AI service:
MODEL_NAME=
ALLOWED_ORIGINS=
```

1. Build and test locally.
2. Create production environment variables.
3. Build images.
4. Run health checks.
5. Deploy backend/AI/database.
6. Deploy frontend with correct API URL.
7. Verify auth and critical workflows.
8. Document deployed URLs and rollback steps.

27. Git/GitHub Strategy

Recommended branches: main for stable releases, development for ongoing integration, feature branches for larger modules.

Commit examples

```
feat: add placement drive module
fix: prevent duplicate applications
test: add eligibility tests
docs: update API documentation
chore: configure docker compose
```

Checkpoint tags

- v0.1-auth
- v0.2-core-placement
- v0.3-frontend
- v0.4-ai
- v1.0-production

Never let a large unfinished change become the only copy of your work. Commit after each stable milestone.

28. Copilot Token-Limit and Context Recovery System

This section is specifically designed so a new Copilot agent can continue even after the previous agent's context is lost.

Required project-memory files

```
docs/AI_CONTEXT.md
docs/PROGRESS.md
docs/DECISIONS.md
docs/API.md
docs/DATABASE.md
docs/ARCHITECTURE.md
docs/tasks/TASK-XXX.md
```

AI_CONTEXT.md template

```
# Current Project Context Project: HireFlow Current phase: Completed: Current task: Important files: Database changes: API changes: Environment variables: Known errors: Last successful test: Next task: Do not change:
```

New-agent master prompt

```
You are continuing the HireFlow project. First read README.md and the relevant docs, especially AI_CONTEXT.md, PROGRESS.md, ARCHITECTURE.md, DATABASE.md and the current task file. Inspect the existing code before changing anything. Do not redesign unrelated modules. Implement only the current task, run relevant tests/checks, fix errors, and update PROGRESS.md and AI_CONTEXT.md before finishing.
```

If an agent reaches its limit, commit the current work if it is in a safe state, update the context files, then start a fresh agent with the prompt above.

29. Complete 30-Day Development Plan

Day	Work	Deliverable
1	Requirements, architecture, docs skeleton	REQUIREMENTS + ARCHITECTURE + AI_CONTEXT
2	Initialize React, Express, MongoDB, FastAPI, Git	Working dev environment
3	Database schemas/indexes	Models + DATABASE.md
4	Backend structure/error handling	Express foundation
5	Authentication	Register/login/me
6	RBAC	Role protection
7	Tests + checkpoint	v0.1-auth
8	Student module	Profile APIs
9	Company module	Company APIs
10	Jobs	Job APIs/search
11	Eligibility	Rules + tests
12	Applications	Application APIs
13	Recruiter APIs	Applicants + shortlist
14	Backend checkpoint	v0.2-core
15	Frontend architecture	Routes/layout/services
16	Auth UI	Login/register/protected routes
17	Student dashboard	Dashboard
18	Profile/resume UI	Profile
19	Job portal	Search/filter/apply
20	Application tracking	Timeline
21	Recruiter UI	Jobs/applicants
22	Officer UI	Dashboard/students/drives
23	Frontend polish	v0.3-frontend
24	AI service foundation	FastAPI endpoints
25	Resume parser	PDF → structured data
26	Semantic matching	Match + missing skills
27	Skill gap + questions	AI preparation
28	Notifications + analytics	Cross-role completion
29	Integration + testing	End-to-end verification
30	Docker + deployment + README	v1.0-production

30. Definition of Done

A feature is done only when the API, database, UI, permissions, error states, tests and documentation are all updated.

- Three role dashboards
- JWT + RBAC
- Student profile + resume
- Company management
- Placement drives
- Configurable eligibility
- Application lifecycle
- Interview scheduling
- Notifications
- Officer analytics
- Recruiter review/shortlist
- Resume parsing
- Semantic matching
- Skill gap
- Interview questions
- OpenAPI docs
- Tests
- Security review
- Docker
- Deployment
- README
- Architecture diagram
- Git checkpoints
- AI_CONTEXT + PROGRESS files

Final demo

1. Officer creates company/drive and eligibility rules.
2. Show eligible students.
3. Student logs in and views drive.
4. Student uploads resume and sees AI match/skill gap.
5. Student applies.
6. Recruiter reviews and shortlists.
7. Interview is scheduled.
8. Officer views final funnel/analytics.
9. Show architecture, Docker and GitHub documentation.

31. Resume and Interview Positioning

Resume project description

HireFlow — Intelligent College Placement Management & Recruitment Platform
Built a full-stack role-based placement platform using React, Node.js, Express.js and MongoDB for students, placement officers and recruiters. Implemented JWT authentication, eligibility automation, placement-drive management, application/interview workflows and analytics. Integrated a Python/FastAPI AI service for resume parsing, semantic resume-job matching, skill-gap analysis and interview-question generation. Containerized services with Docker and documented REST APIs using OpenAPI.

Questions to prepare

- Why separate Node and Python services?
- How does JWT work?
- How is RBAC enforced?
- Why MongoDB?
- How are duplicate applications prevented?
- Eligibility vs AI matching?
- How are embeddings used?
- How would you evaluate matching?
- What happens if AI is down?
- How would you scale the system?
- How are resumes secured?
- What tests did you write?
- How does Docker help?

32. Future Enhancements

Only add these after the core system is stable.

- Email/SMS notifications
- Calendar integration
- Online assessments
- Coding-test integration
- Offer-letter management
- Advanced reports
- Vector database for larger-scale semantic search
- Recommendation engine
- Multi-tenant college support
- Advanced audit/compliance controls

Final principle: **build the complete placement workflow first, then add AI.** A polished, working platform with useful AI features is stronger than many unfinished AI demos.